# Exercise: Filtering, Sorting, Aggregating, and Binning

Explore how to apply filtering sorting aggregating and binning techniques on sales data. Learn to manipulate data views to meet business needs by completing code snippets for each task, improving your data transformation skills in ETL pipelines.

Suppose we have a table called `sales` that contains information about the sales of various products in a store. Let's see what the table looks like:

main.sql MySQL

```sql
/* Print the first 5 records*/
select *
from sales
limit 5;
```

Run

Sample data

Ask

Our task is to apply filtering, sorting, aggregating, and binning to the table according to the business requirements.

> **Note:** Fill in the missing code snippets to complete the task.

## Filtering

The first task is to filter all the sales that occurred in the month of January. Filter the data and create a `view` with the relevant data.

**main.sql** MySQL

```
/* Create a view of january sales */
create view january_sales as
select *
from sales
 <<InsertHere>>

/* Print view*/
select *
from january_sales;
```

Run

Filtering january sales

## Sorting

Next, we're asked to create a `view` of all transactions that involved electronic products and sort them by price in a descending format.

**main.sql** MySQL

Ask

```
/* Create a view of electornic products */
create view electronics_desc as
select *
from sales
```

```
<<InsertHere>>

/* Print view*/
select *
from electronics_desc;
```

Run

Sorting electronic products

## Aggregating

The business is interested in finding out the total revenue of each product in the sales table. Total revenue can be calculated by multiplying the price of the product by the amount sold and grouping the results to get a sum of the total revenue for each product.

main.sql MySQL

```
/* Create a view of product revenue */
create view product_revenue as
select
    name,
    <<InsertHere>> as total_revenue
from sales
<<InsertHere>>
order by total_revenue desc;


/*Print view*/
select *
from product_revenue;
```

Ask

Aggregating product sales

## Binning

Finally, we're tasked with binning the `product_revenue` view from the previous task and binning each product in either one of the following bins:

- High (greater than 2000)
- Medium (between 400 and 2000)
- Low (less than 400)

main.sql MySQL

```
/* Create a view of total revenue divided into bins*/
create view total_revenue_bins as
select
    name,
    total_revenue,
    <<InsertHere>>
from product_revenue;


/* Print data*/
select *
from total_revenue_bins;
```

Run

Binning total revenue

## Solution codes

Here are the solution codes:

### Filtering

Ask

main.sql MySQL

```
/* Create a view of january sales */
create view january_sales as
select *
from sales
where
    date between '2023-01-01' and '2023-02-01';

/* Print view*/
select *
from january_sales;
```

Run

Solution code: Filtering january sales

- **Lines 5–6:** We filter all sale records between `2023-01-01` (inclusive) to `2023-02-01` (exclusive).

## Sorting

main.sql MySQL

```
/* Create a view of electornic products */
create view electronics_desc as
select *
from sales
where
    category = 'electronics'
order by category desc;

/* Print view*/
select *
from electronics_desc;
```

Run

Ask

Solution code: Sorting electronic products

- **Lines 5–6:** We filter products of the category `electornics` only.

- **Line 7:** We use `order by` to query the products in a descending format.

## Aggregating

main.sql MySQL

```
/* Create a view of product revenue */
create view product_revenue as
select
    name,
    sum(price * amount_sold) as total_revenue
from sales
group by name
order by total_revenue desc;


/*Print view*/
select *
from product_revenue;
```

Run

Solution code: Aggregating product sales

- **Line 5:** We calculate total revenue by multiplying `price` with `amount_sold`.
- **Line 7:** We group products by `name`, allowing us to use aggregation functions such as `sum`.

## Binning

main.sql MySQL

```
/* Create a view of total revenue divided into bins*/
create view total_revenue_bins as
select
    name,
    total_revenue,
    case
        when total_revenue >= 2000 then 'High'
        when total_revenue between 400 and 2000 then 'Medium'
        else 'Low'
```

Ask

```
    end as revenue_bin
from product_revenue;
```

Run

Solution code: Binning total revenue

- **Lines 6–10:** We apply the logic to bin product revenues using a `case` statement.

Ask